

In [8]: `%pip install opencv-python numpy matplotlib`

```
Requirement already satisfied: opencv-python in /usr/local/python/3.12.1/lib/
python3.12/site-packages (5.0.0.93)
Requirement already satisfied: numpy in /usr/local/python/3.12.1/lib/python3.
12/site-packages (2.5.2)
Requirement already satisfied: matplotlib in /usr/local/python/3.12.1/lib/pyt
hon3.12/site-packages (3.11.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/python/3.12.1/l
ib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in /usr/local/python/3.12.1/lib/p
ython3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/python/3.12.1/
lib/python3.12/site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/python/3.12.1/
lib/python3.12/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/lib/
python3.12/site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=9 in /usr/local/python/3.12.1/lib/pyth
on3.12/site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /usr/local/python/3.12.1/lib/p
ython3.12/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /home/codespace/.loca
l/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/codespace/.local/lib/python
3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

[notice] A new release of pip is available: 26.0.1 -> 26.2.1

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

In [9]: `import cv2
import numpy as np
import matplotlib.pyplot as plt`

```
print("OpenCV version:", cv2.__version__)
print("All libraries imported successfully!")
```

OpenCV version: 5.0.0

All libraries imported successfully!

In [10]: `# Load the image
image = cv2.imread("image.png")

Check whether image is loaded
if image is None:
 print("✗ Image not found")
else:
 print("✓ Image loaded successfully!")
 print("Image shape:", image.shape)

Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

Display original image`

```
plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

✓ Image loaded successfully!

Image shape: (1024, 1024, 3)

Original Image



In [11]: *# Step 3: Apply Gaussian Blur*

```
# Apply Gaussian Blur
gaussian_blur = cv2.GaussianBlur(image, (5, 5), 0)

# Convert blurred image from BGR to RGB
gaussian_rgb = cv2.cvtColor(gaussian_blur, cv2.COLOR_BGR2RGB)

# Display original and Gaussian filtered image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gaussian_rgb)
```

```
plt.title("Gaussian Blur")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Gaussian Blur



In [12]: *# Step 4: Apply Median Filter*

```
median_filter = cv2.medianBlur(image, 5)

# Convert BGR to RGB
median_rgb = cv2.cvtColor(median_filter, cv2.COLOR_BGR2RGB)

# Display original and median filtered image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(median_rgb)
plt.title("Median Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Median Filter



```
In [13]: # Step 5: Apply Average (Mean) Filter

average_filter = cv2.blur(image, (5, 5))

# Convert BGR to RGB
average_rgb = cv2.cvtColor(average_filter, cv2.COLOR_BGR2RGB)

# Display original and average filtered image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(average_rgb)
plt.title("Average (Mean) Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Average (Mean) Filter



```
In [14]: # Step 6: Apply Laplacian Filter

# Convert image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Laplacian filter
laplacian = cv2.Laplacian(gray_image, cv2.CV_64F)

# Convert result to uint8 for display
laplacian = cv2.convertScaleAbs(laplacian)

# Display original and Laplacian filtered image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Grayscale Image



Laplacian Filter



```
In [15]: # Step 7: Sobel Edge Detection

# Sobel X - detects vertical edges
sobel_x = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)

# Sobel Y - detects horizontal edges
sobel_y = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
sobel_y = cv2.convertScaleAbs(sobel_y)

# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X - Vertical Edges")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y - Horizontal Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```



In [16]: *# Step 8: Compare all filtering techniques*

```
plt.figure(figsize=(15, 10))

# 1. Original
plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# 2. Gaussian
plt.subplot(2, 3, 2)
plt.imshow(gaussian_rgb)
plt.title("Gaussian Blur")
plt.axis("off")

# 3. Median
plt.subplot(2, 3, 3)
plt.imshow(median_rgb)
plt.title("Median Filter")
plt.axis("off")

# 4. Average
plt.subplot(2, 3, 4)
plt.imshow(average_rgb)
plt.title("Average Filter")
plt.axis("off")

# 5. Laplacian
plt.subplot(2, 3, 5)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

# 6. Sobel X + Y
sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

plt.subplot(2, 3, 6)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Sobel Combined")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



```
In [17]: import time

# Gaussian timing
start = time.time()
cv2.GaussianBlur(image, (5, 5), 0)
gaussian_time = time.time() - start

# Median timing
start = time.time()
cv2.medianBlur(image, 5)
median_time = time.time() - start

# Average timing
start = time.time()
cv2.blur(image, (5, 5))
average_time = time.time() - start

print("Gaussian Filter Time:", gaussian_time, "seconds")
print("Median Filter Time:", median_time, "seconds")
print("Average Filter Time:", average_time, "seconds")
```

```
Gaussian Filter Time: 0.006658792495727539 seconds
Median Filter Time: 0.007248401641845703 seconds
Average Filter Time: 0.0031452178955078125 seconds
```

```
In [18]: # Step 10: Final comparison of filtering techniques

filters = {
```

```

    "Gaussian Filter": gaussian_blur,
    "Median Filter": median_filter,
    "Average Filter": average_filter,
    "Laplacian Filter": laplacian,
    "Sobel Combined": sobel_combined
}

print("===== FINAL OBSERVATIONS =====\n")

print("1. Gaussian Filter:")
print("    - Smooths the image and reduces noise.")
print("    - Preserves important structures relatively well.\n")

print("2. Median Filter:")
print("    - Effective for removing salt-and-pepper noise.")
print("    - Preserves edges better than simple averaging.\n")

print("3. Average Filter:")
print("    - Provides simple image smoothing.")
print("    - Can blur edges and fine details.\n")

print("4. Laplacian Filter:")
print("    - Highlights edges and fine image details.")
print("    - Sensitive to noise.\n")

print("5. Sobel Filter:")
print("    - Detects horizontal and vertical edges.")
print("    - Useful for feature and boundary detection.")

```

===== FINAL OBSERVATIONS =====

1. Gaussian Filter:
 - Smooths the image and reduces noise.
 - Preserves important structures relatively well.
2. Median Filter:
 - Effective for removing salt-and-pepper noise.
 - Preserves edges better than simple averaging.
3. Average Filter:
 - Provides simple image smoothing.
 - Can blur edges and fine details.
4. Laplacian Filter:
 - Highlights edges and fine image details.
 - Sensitive to noise.
5. Sobel Filter:
 - Detects horizontal and vertical edges.
 - Useful for feature and boundary detection.

Conclusion

Spatial filtering techniques were successfully implemented using Python and OpenCV. Low-pass filters such as Gaussian, Median, and Average filters were used for image smoothing and noise reduction. High-pass filters such as Laplacian and Sobel were

used for edge and detail enhancement. The results showed that Gaussian filtering provides smooth noise reduction, Median filtering is effective for salt-and-pepper noise, while Laplacian and Sobel filters are useful for detecting edges and fine details. Thus, spatial filtering is an important preprocessing technique in digital image processing and computer vision.